

TypeORM 数据库实践

博客系统项目

版权声明

本内容版权属杭州饥人谷教育科技有限公司（简称饥人谷）所有。

任何媒体、网站或个人未经本网协议授权不得转载、链接、转贴，或以其他方式复制、发布和发表。

已获得饥人谷授权的媒体、网站或个人在使用时须注明「资料来源：饥人谷」。

对于违反者，饥人谷将依法追究其责任。

联系方式

如果你想要购买本课程

请微信联系 [xiedaimala02](#) 或 [xiedaimala03](#)

如果你发现有人盗用本课程

请微信联系 [xiedaimala02](#) 或 [xiedaimala03](#)

初始代码

- 仓库地址

- ✓ <https://github.com/FrankFang/nextjs-typeorm-1>
- ✓ 如果该地址 404 了，说明我在反盗版，请在群里问新链接

- 代码使用步骤

- ✓ `cd Jirengu`
- ✓ `git clone git@...` (请自行拷贝地址)
- ✓ `cd nextjs-typeorm-1`
- ✓ `yarn install`
- ✓ `yarn dev`

TypeORM 简介

- 版本

- ✓ 0.2.25, 还没有发布 v1.0.0

- 特性

- ✓ 默认支持 TypeScript
- ✓ 我本打算用 Sequelize.js, 发现它对 TS 支持不够好
- ✓ 支持关联 (Associations)
- ✓ 支持事务 (Transaction)
- ✓ 支持数据库迁移 (Migration)

启动数据库 postgresql

- 创建数据目录 (Windows Toolbox 省)

- ✓ 在项目目录里创建 blog-data 目录
- ✓ .gitignore 里添加 /blog-data/

- 启动 PostgreSQL

- ✓ 一句命令启动 pg:

```
docker run -v "$PWD/blog-data":/var/lib/postgresql/data -p 5432:5432 -e POSTGRES_USER=blog -e POSTGRES_HOST_AUTH_METHOD=trust -d postgres:12.2
```

- ✓ 如果报错，可以看这篇[教程](#)，或将报错复制到 Google
- ✓ 如果下载慢，先搜[阿里云 Docker 镜像](#)（要登录阿里云），再搜 Docker 加速或 [Docker Toolbox 加速](#)
- ✓ docker ps -a 这句话可以查看容器运行状态
- ✓ docker logs 容器id 这句话可以查看启动日志

验证 pg

- 进入 docker 容器

- ✓ `docker exec -it 容器id bash`

- 进入 pg 命令行

- ✓ `psql -U blog -W`
- ✓ 我设置了不需要密码，所以直接回车即可
- ✓ 如果你需要密码，可在 `docker run` 选项里的 `-e POSTGRES_HOST_AUTH_METHOD=trust` 替换成 `-e POSTGRES_PASSWORD=123456`

- 执行 pg 命令

- ✓ `\l` 用于 list databases，目前有一个 blog 数据库
- ✓ `\c` 用于 connect to a database
- ✓ `\dt` 用于 display tables，目前我们没有 tables

创建数据库

- 用 SQL 来创建数据库

- ✓ 因为 TypeORM 没有提供单纯创建数据库的 API（差评）

```
CREATE DATABASE xxx ENCODING 'UTF8'  
LC_COLLATE 'en_US.utf8' LC_CTYPE 'en_US.utf8';
```

- ✓ 我们需要创建三个数据库：开发、测试、生产
- ✓ 对应英文：development、test、production
- ✓ 最终结果得到三个数据库

安装 TypeORM

• 步骤

- ✓ 打开官网，点击 Getting Started
- ✓ 找能运行的代码

• 结果

- ✓ 先 `git commit . -m 'install pg'` 提交代码
- ✓ 再安装依赖：`yarn add typeorm reflect-metadata @types/node pg`
- ✓ 然后修改 `tsconfig`
- ✓ 注意，这里要 `git commit . -m 'add typeorm'` 一下
- ✓ 然后 `npx typeorm init --database postgres`
- ✓ **重要：**然后把一些文件还原一下（WebStorm或命令行）
- ✓ 最后修改 `ormconfig.json` 和 `src/index.ts`

如果 `npx typeorm` 失败就改成 `./node_modules/.bin/typeorm`

吐槽

- 如何运行 TypeScript

- ✓ Next.js 默认使用 babel 来将 TS 编译为 JS（内置功能）
- ✓ TypeORM 推荐使用 ts-node 来编译（没有内置）
- ✓ babel 和 ts-node 对 TS 的支持并非完全一致
- ✓ 所以必须进行统一，全都用 babel

连接数据库

- 步骤

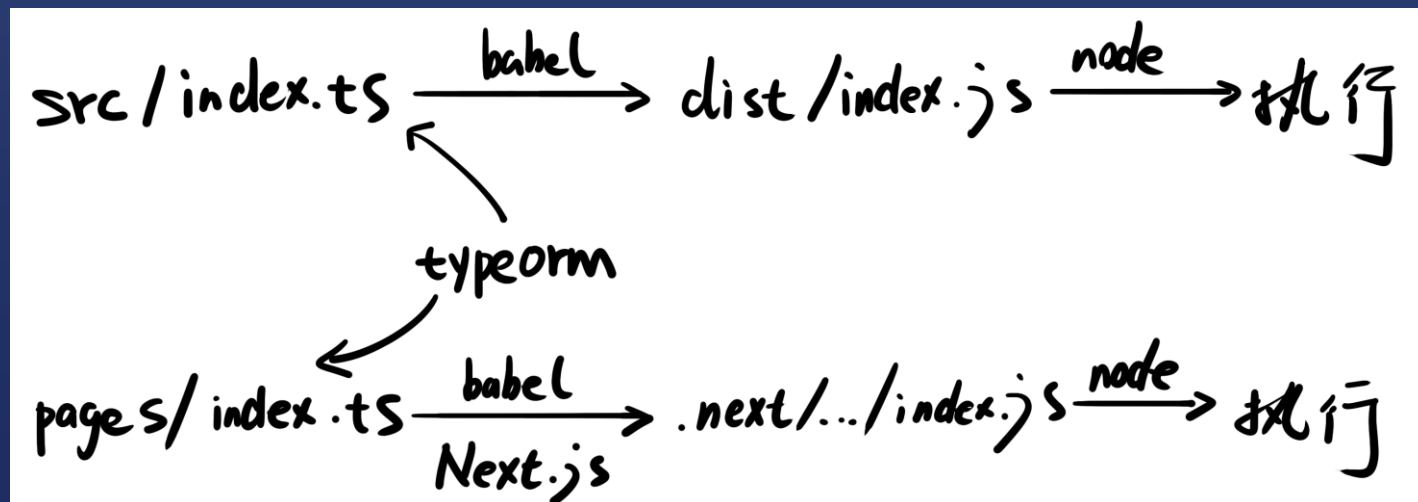
- ✓ 选做：升级 Node.js 到 v14
- ✓ 安装 @babel/cli
- ✓ `npx babel ./src --out-dir dist --extensions ".ts,.tsx"`
- ✓ 根据错误提示搜索答案
- ✓ `yarn add --dev @babel/plugin-proposal-decorators`
- ✓ 去 Next.js 官网查看 .babelrc 默认配置，复制
- ✓ 创建 .babelrc，添加插件
- ✓ 重新运行刚刚失败的命令
- ✓ 得到 dist 里面的 JS，运行 `node dist/index.js`
- ✓ 根据错误提示再加上数小时的搜索，修改 ormconfig
- ✓ 重新运行 `node dist/index.js`
- ✓ 成功看见 connection 对象即为成功

workflow总结

• 步骤

- ✓ 统一让 Next.js 和 TypeORM 使用 babel 翻译 TS
- ✓ 每次修改 src 的 TS 代码后，翻译为 dist 里的 JS
- ✓ 使用 node 运行 dist 里的 JS，执行 TypeORM 任务
- ✓ 也可使用 Next.js 执行 TypeORM 任务（后面讲）

• 图示



重要配置：禁用 sync

- ormconfig

- ✓ "synchronize": true, => 改为 false,
- ✓ 如果为 true，那么在连接数据库时，typeorm 会自动根据 entity 目录来修改数据表
- ✓ 假设 entity 里面有 User，就会自动创建 User 表

- 看起来很方便，为什么要禁用

- ✓ 因为 sync 功能可能会在我们修改 User 时直接删数据
- ✓ 这种行为绝对不能发生在生产环境
- ✓ 所以我们要一开始就杜绝 sync 功能

创建表（通过 migration）

- ✓ 请事先 git commit，否则出错了不好撤销
- ✓ 请确保目前没有表，如果有就 drop table xxx

• posts 表

- ✓ npx typeorm migration:create -n CreatePost
- ✓ 得到 src/migrations/{TIMESTAMP}-CreatePost.ts
- ✓ 参考[文档](#)，写 up 函数
- ✓ 为了方便撤销，写 down 函数

• 运行 migration（数据迁移）

- ✓ npx babel ./src --out-dir dist --extensions ".ts,.tsx"
- ✓ npx typeorm migration:run
- ✓ npx typeorm migration:revert

吐槽

- **npx 在 Windows 上极慢**

- ✓ 建议把所有命令都写在 package.json 里面
- ✓ 默认会先从 node_modules/.bin 里面找命令
- ✓ 同时，这也可以防止你忘了命令
- ✓ 请保持这个习惯

- **每次都要运行 babel 不傻吗？**

- ✓ npx babel --help 可以看到有 -w 选项
- ✓ 我们让 yarn dev 同时运行 next dev 和 babel -w 即可
- ✓ 但是 Windows 不支持 & 操作，只有 Linux / Mac 支持
- ✓ 于是我们搜索 npm run tasks in parallel
- ✓ 安装 concurrently 代替 & 操作
- ✓ 前端工具链真的烦死人！什么都不自带！

数据映射到实体

- 背景

- ✓ 刚刚只是在数据库里创建了 Post，代码如何读写 Post 呢
- ✓ 答案：将数据映射到 Entity（实体）
- ✓ 命令：typeorm entity:create -n Post

- 知识点

- ✓ @PrimaryGeneratedColumn('increment')
- ✓ @Column('varchar')
- ✓ @Column('text')

- 如何使用实体

- ✓ EntityManager 或 Repository
- ✓ 这只是两种不同的封装思路而已，需要灵活使用

EntityManager API

- 举例

- ✓ `await manager.find(User, { name: "frank" });`
- ✓ `await manager.create(User, { name: '...' })`
- ✓ `await manager.save(user1)`
- ✓ `await manager.save([user1, user2, user3])`
- ✓ `await manager.remove(user1)`
- ✓ `await manager.update(User, 1, { name: 'frank' });`
- ✓ `await manager.delete(User, 1);`
- ✓ `await manager.findOne(User, 1);`

- 封装思路

- ✓ 把所有操作都放在 `manager` 上
- ✓ 把 `User` 类、`user1` 对象和其他参数传给 `manager`

Repository API

- 举例

- ✓ `const userRepository = getRepository(User);`
- ✓ `await userRepository.findOne(1)`
- ✓ `await userRepository.save(user)`

- 封装思路

- ✓ 先通过 User 构造一个 repo 对象
- ✓ 这个 repo 对象就只操作 User 表了

- 特色

- ✓ TreeRepository 和 `MongoRepository`
- ✓ 目前用不到这两个功能，所以就先不用 repo API 吧

测试一下 Post 实体

```
const posts = await connection.manager.find(Post);
console.log(posts);
const p = new Post();
p.title = 'post 1';
p.content = 'my first post';
await connection.manager.save(p);
const posts2 = await connection.manager.find(Post);
console.log(posts2);
```

- 输出

```
$ node dist/index.js
```

```
[]
```

```
[ Post { id: 1, title: 'post 1', content: 'my  
first post' } ]
```

总结

- migration 数据迁移

- ✓ 用来对数据库进行升级和降级

- entity 实体

- ✓ 用类和对象来操作数据表和数据行

- connection 连接

- ✓ 一个数据库连接，默认最多 10 个连接
- ✓ 这种模式叫做连接池，可以参考[这篇文章](#)

- manager / repo

- ✓ 两种 API 封装风格，用于操作 entity

Seed

- 也叫数据填充

- ✓ 现在数据库有了，数据表 posts 也有了，但是没有数据
- ✓ 我们可以通过 seed 脚本来构造数据
- ✓ 这比我们写一个网页加一个表单造数据简单得多
- ✓ 一般不在生产环境运行 seed 脚本

- 好处

- ✓ 方便你的同事快速运行项目
- ✓ 不然你就会被吐槽：请问你创建的这个表怎么填数据
- ✓ 为了防止被新同事烦死，你应该写 seed 脚本（务必！）

数据库搞定，开始开发

先设计数据再做开发，前端转后端需要适应这一点

下节课见

开始最简单的增删改查